Performance Tuning

Optimize T3000 system performance and responsiveness.

Database Optimization

Retry Settings

Current configuration:

- Max retries: 10
- Base delay: 100ms
- Exponential backoff
- Total wait: ~51 seconds

Data Retention

- Real-time data: 24 hours
- Historical data: 1 year
- Archive older data to reduce database size

Network Tuning

Polling Rates

Recommended:

- Critical points: 5 seconds
- Normal points: 10 seconds
- Slow-changing: 60 seconds

Concurrent Requests

- Limit: 10 concurrent requests
- Queue additional requests
- Batch operations when possible

Browser Optimization

Chart Performance

- Limit charts to 10 maximum
- Use appropriate time ranges
- Reduce displayed points
- Close unused tabs

Memory Management

- Clear cache periodically
- Reload page if sluggish
- Use latest browser version

Server Optimization

API Server

- Monitor CPU/memory usage
- Restart if memory leak
- Use release build (not debug)
- Check logs for errors

Database

- Vacuum SQLite regularly
- Optimize indexes
- Archive old data
- Monitor file size

Advanced Performance Tuning

Database Optimization

SQLite Performance Tuning

SQL

```sql
-- Enable Write-Ahead Logging (WAL)
PRAGMA journal_mode = WAL;

-- Increase cache size (in pages, 1 page = 4KB)
PRAGMA cache_size = -64000;  -- 64MB cache

-- Use memory-mapped I/O
PRAGMA mmap_size = 268435456;  -- 256MB

-- Optimize synchronization
PRAGMA synchronous = NORMAL;  -- Faster than FULL, still safe

-- Set optimal page size
PRAGMA page_size = 4096;

-- Enable auto-vacuum
PRAGMA auto_vacuum = INCREMENTAL;
```

Query Optimization

TYPESCRIPT

```typescript
// ✅ Good: Use indexes
// Create index on frequently queried columns
await db.exec(`
  CREATE INDEX IF NOT EXISTS idx_trend_logs_device_point_time
  ON trend_logs(device_id, point, timestamp DESC)
`);

// Query will use index
const data = await db.all(`
  SELECT * FROM trend_logs
  WHERE device_id = ? AND point = ? AND timestamp > ?
  ORDER BY timestamp DESC
  LIMIT 1000
`, [deviceId, point, startTime]);

// ❌ Bad: Full table scan
const data = await db.all(`
  SELECT * FROM trend_logs
  WHERE CAST(device_id AS TEXT) = '${deviceId}'
`);
```

Connection Pooling

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class DatabasePool {
  private pool: Database.Database[] = [];
  private available: Database.Database[] = [];
  private readonly maxConnections: number;

  constructor(dbPath: string, maxConnections = 5) {
    this.maxConnections = maxConnections;

    for (let i = 0; i < maxConnections; i++) {
      const db = new Database(dbPath);
      db.pragma('journal_mode = WAL');
      db.pragma('cache_size = -64000');
      this.pool.push(db);
      this.available.push(db);
    }
  }

  async acquire(): Promise<Database.Database> {
    while (this.available.length === 0) {
      await new Promise(resolve => setTimeout(resolve, 10));
    }
    return this.available.pop()!;
  }

  release(db: Database.Database): void {
    this.available.push(db);
  }

  async withConnection<T>(
    fn: (db: Database.Database) => Promise<T>
  ): Promise<T> {
    const db = await this.acquire();
    try {
      return await fn(db);
    } finally {
      this.release(db);
```

```
    }
  }
}
```

Batch Operations

TYPESCRIPT

```typescript
// ✅ Good: Batch insert
async function batchInsertTrendData(
  data: TrendData[]
): Promise<void> {
  const db = await dbPool.acquire();

  const insert = db.prepare(`
    INSERT INTO trend_logs (device_id, point, timestamp, value)
    VALUES (?, ?, ?, ?)
  `);

  const transaction = db.transaction((records: TrendData[]) => {
    for (const record of records) {
      insert.run(record.deviceId, record.point, record.timestamp, record.value);
    }
  });

  transaction(data);
  dbPool.release(db);
}

// Process 1000 records in one transaction
await batchInsertTrendData(records);  // ~10ms

// ❌ Bad: Individual inserts
for (const record of records) {
  await db.run(
    'INSERT INTO trend_logs VALUES (?, ?, ?, ?)',
    [record.deviceId, record.point, record.timestamp, record.value]
  );
}  // ~1000ms for 1000 records
```

Caching Strategies

Multi-Level Cache

TYPESCRIPT

```typescript
class MultiLevelCache {
  private l1Cache = new Map<string, CacheEntry>();  // Memory
  private l2Cache: LocalForage;  // IndexedDB
  private maxL1Size = 1000;

  async get(key: string): Promise<any | null> {
    // Check L1 (memory)
    const l1Entry = this.l1Cache.get(key);
    if (l1Entry && l1Entry.expires > Date.now()) {
      return l1Entry.value;
    }

    // Check L2 (IndexedDB)
    const l2Entry = await this.l2Cache.getItem<CacheEntry>(key);
    if (l2Entry && l2Entry.expires > Date.now()) {
      // Promote to L1
      this.l1Cache.set(key, l2Entry);
      return l2Entry.value;
    }

    return null;
  }

  async set(key: string, value: any, ttl: number): Promise<void> {
    const entry: CacheEntry = {
      value,
      expires: Date.now() + ttl
    };

    // Store in L1
    this.l1Cache.set(key, entry);

    // Evict if L1 too large
    if (this.l1Cache.size > this.maxL1Size) {
      const firstKey = this.l1Cache.keys().next().value;
      this.l1Cache.delete(firstKey);
```

```
    }

    // Store in L2
    await this.l2Cache.setItem(key, entry);
  }
}
```

Cache Warming

TYPESCRIPT

```
class CacheWarmer {
  async warmDeviceCache(deviceIds: number[]): Promise<void> {
    const promises = deviceIds.map(async (deviceId) => {
      // Fetch and cache device info
      const device = await fetch(`/api/devices/${deviceId}`).then(r => r.json());
      cache.set(`device:${deviceId}`, device, 300000);  // 5 min

      // Fetch and cache points
      const points = await fetch(`/api/devices/${deviceId}/points`).then(r => r.json());
      cache.set(`points:${deviceId}`, points, 300000);
    });

    await Promise.all(promises);
  }
}
```

Network Optimization

Request Deduplication

TYPESCRIPT

```typescript
class RequestDeduplicator {
  private pending = new Map<string, Promise<any>>();

  async fetch<T>(key: string, fetcher: () => Promise<T>): Promise<T> {
    // Return existing request if pending
    if (this.pending.has(key)) {
      return this.pending.get(key)!;
    }

    // Create new request
    const promise = fetcher().finally(() => {
      this.pending.delete(key);
    });

    this.pending.set(key, promise);
    return promise;
  }
}

const deduper = new RequestDeduplicator();

// Multiple calls will only trigger one fetch
const [data1, data2, data3] = await Promise.all([
  deduper.fetch('device:389001', () => fetch('/api/devices/389001').then(r => r.json())),
  deduper.fetch('device:389001', () => fetch('/api/devices/389001').then(r => r.json())),
  deduper.fetch('device:389001', () => fetch('/api/devices/389001').then(r => r.json()))
]);  // Only 1 network request made
```

Request Batching

TYPESCRIPT

```typescript
class RequestBatcher {
  private queue: BatchRequest[] = [];
  private timer: NodeJS.Timeout | null = null;
  private batchSize = 10;
  private batchDelay = 50;  // ms

  async add(request: BatchRequest): Promise<any> {
    return new Promise((resolve, reject) => {
      this.queue.push({ ...request, resolve, reject });

      if (this.queue.length >= this.batchSize) {
        this.flush();
      } else if (!this.timer) {
        this.timer = setTimeout(() => this.flush(), this.batchDelay);
      }
    });
  }

  private async flush(): Promise<void> {
    if (this.timer) {
      clearTimeout(this.timer);
      this.timer = null;
    }

    if (this.queue.length === 0) return;

    const batch = this.queue.splice(0);

    try {
      const response = await fetch('/api/batch', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          requests: batch.map(r => ({ endpoint: r.endpoint, params: r.params }))
        })
      });

      const results = await response.json();
```

```
      batch.forEach((req, i) => {
        if (results[i].error) {
          req.reject(new Error(results[i].error));
        } else {
          req.resolve(results[i].data);
        }
      });
    } catch (error) {
      batch.forEach(req => req.reject(error));
    }
  }
}
```

HTTP/2 Server Push

TYPESCRIPT

```
// Backend (Express + SPDY)
import spdy from 'spdy';

app.get('/devices/:id', (req, res) => {
  // Push related resources
  const stream = res.push('/api/devices/' + req.params.id + '/points', {
    request: { accept: 'application/json' },
    response: { 'content-type': 'application/json' }
  });

  stream.end(JSON.stringify(pointsData));

  res.json(deviceData);
});
```

Frontend Optimization

Virtual Scrolling

VUE

```vue
<template>
  <virtual-scroller
    :items="devices"
    :item-height="50"
    :buffer="200"
  >
    <template #default="{ item }">
      <DeviceRow :device="item" />
    </template>
  </virtual-scroller>
</template>

<script setup lang="ts">
// Only renders visible items + buffer
// Handles 10,000+ items smoothly
</script>
```

Lazy Loading Components

TYPESCRIPT

```typescript
import { defineAsyncComponent } from 'vue';

const HeavyChart = defineAsyncComponent(() =>
  import('./components/HeavyChart.vue')
);

const DeviceDetails = defineAsyncComponent({
  loader: () => import('./components/DeviceDetails.vue'),
  loadingComponent: LoadingSpinner,
  delay: 200,  // Show loading after 200ms
  timeout: 10000  // Error after 10s
});
```

Debouncing and Throttling

TYPESCRIPT

```typescript
import { debounce, throttle } from 'lodash-es';

// Debounce: Wait for user to stop typing
const searchDevices = debounce(async (query: string) => {
  const results = await fetch(`/api/devices/search?q=${query}`);
  // ...
}, 300);

// Throttle: Limit execution rate
const updateChart = throttle((data: number[]) => {
  chart.setData(data);
}, 1000);  // Max once per second
```

Memory Management

Object Pooling

TYPESCRIPT

```typescript
class ObjectPool<T> {
  private available: T[] = [];
  private factory: () => T;
  private reset: (obj: T) => void;

  constructor(factory: () => T, reset: (obj: T) => void, initialSize = 10) {
    this.factory = factory;
    this.reset = reset;

    for (let i = 0; i < initialSize; i++) {
      this.available.push(factory());
    }
  }

  acquire(): T {
    return this.available.pop() || this.factory();
  }

  release(obj: T): void {
    this.reset(obj);
    this.available.push(obj);
  }
}

// Usage: Reuse objects instead of creating new ones
const pointDataPool = new ObjectPool<PointData>(
  () => ({ deviceId: 0, point: '', value: 0, timestamp: 0 }),
  (obj) => { obj.deviceId = 0; obj.point = ''; obj.value = 0; obj.timestamp = 0; }
);
```

Garbage Collection Hints

TYPESCRIPT

```typescript
class LargeDataProcessor {
  private cache = new WeakMap<object, ProcessedData>();

  process(rawData: object): ProcessedData {
    // Check cache
    const cached = this.cache.get(rawData);
    if (cached) return cached;

    // Process and cache
    const processed = this.heavyProcessing(rawData);
    this.cache.set(rawData, processed);  // Auto-GC'd when rawData is GC'd

    return processed;
  }
}
```

Monitoring and Profiling

Performance Metrics

TYPESCRIPT

```typescript
class PerformanceMetrics {
  recordMetric(name: string, value: number, tags?: Record<string, string>) {
    // Send to monitoring service
    fetch('/api/metrics', {
      method: 'POST',
      body: JSON.stringify({ name, value, tags, timestamp: Date.now() })
    });
  }

  measureAsync<T>(name: string, fn: () => Promise<T>): Promise<T> {
    const start = performance.now();
    return fn().finally(() => {
      this.recordMetric(name, performance.now() - start);
    });
  }
}

// Usage
await metrics.measureAsync('fetch-device-data', () =>
  fetch('/api/devices/389001').then(r => r.json())
);
```

Next Steps

- Best Practices
- Troubleshooting
- FAQ